15 Working with Spring Boot Actuator

This chapter covers

- Enabling Actuator in Spring Boot projects
- Exploring Actuator endpoints
- Customizing Actuator
- Securing Actuator

Have you ever tried to guess what's inside a wrapped gift? You shake it, weigh it, and measure it. And you might even have a solid idea as to what's inside. But until you open it up, there's no way of knowing for sure.

A running application is kind of like a wrapped gift. You can poke at it and make reasonable guesses as to what's going on under the covers. But how can you know for sure? If only there were some way that you could peek inside a running application, see how it's behaving, check on its health, and maybe even trigger operations that influence how it runs!

In this chapter, we're going to explore Spring Boot Actuator. Actuator offers production-ready features such as monitoring and metrics to Spring Boot applications. Actuator's features are provided by way of several endpoints, which are made available over HTTP as well as through JMX MBeans. This chapter focuses primarily on HTTP endpoints, saving JMX endpoints for chapter 18.

15.1 Introducing Actuator

In a machine, an actuator is a component that's responsible for controlling and moving a mechanism. In a Spring Boot application, the Spring Boot Actuator plays that same role, enabling us to see inside of a running application and, to some degree, control how the application behaves.

Using endpoints exposed by Actuator, we can ask things about the internal state of a running Spring Boot application, such as the following:

- What configuration properties are available in the application environment?
- What are the logging levels of various packages in the application?
- How much memory is being consumed by the application?
- How many times has a given HTTP endpoint been requested?
- What is the health of the application and any external services it coordinates with?

To enable Actuator in a Spring Boot application, you simply need to add Actuator's starter dependency to your build. In any Spring Boot application Maven pom.xml file, the following `<dependency>` entry does the trick:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Once the Actuator starter is part of the project build, the application will be equipped with several out-of-the-box Actuator endpoints, including those described in table 15.1.

Table 15.1 Actuator endpoints for peeking inside and manipulating the state of a running Spring Boot application

HTTP method	Path	Description
GET	/auditevents	Produces a report of any audit events that have been fired
GET	/beans	Describes all the beans in the Spring application context
GET	/conditions	Produces a report of autoconfiguration conditions that either passed or failed, leading to the beans created in the application context
GET	/configprops	Describes all configuration properties along with the current values
GET, POST, DELETE	/env	Produces a report of all property sources and their properties available to the Spring application
GET	/env/{toMatch}	Describes the value of a single environment property
GET	/health	Returns the aggregate health of the application and (possibly) the health of external dependent applications
GET	/heapdump	Downloads a heap dump
GET	/httptrace	Produces a trace of the most recent 100 requests
GET	/info	Returns any developer-defined information about the application
GET	/loggers	Produces a list of packages in the application along with their configured and effective logging levels
GET, POST	/loggers/{name}	Returns the configured and effective logging level of a given logger; the effective logging level can be set with a POST request
GET	/mappings	Produces a report of all HTTP mappings and their corresponding handler methods
GET	/metrics	Returns a list of all metrics categories
GET	/metrics/{name}	Returns a multidimensional set of values for a given metric
GET	/scheduledtasks	Lists all scheduled tasks
GET	/threaddump	Returns a report of all application threads

In addition to HTTP endpoints, all of the Actuator endpoints in table 15.1, with the lone exception of /heapdump, are also exposed as JMX MBeans. We'll look at the JMX side of Actuator in chapter 17.

15.1.1 Configuring Actuator's base path

By default, the paths for all the endpoints shown in table 15.1 are prefixed with /actuator. This mean that, for example, if you wish to retrieve health information about your application from Actuator, then issuing a `GET` request for /actuator/health will return the information you need.

The Actuator prefix path can be changed by setting the `management.endpoint.web.base-path` property. For example, if you'd rather the prefix be /management, you would set the `management.endpoints.web.base-path` property like this:

```
management:
  endpoints:
    web:
      base-path: /management
```

With this property set as shown, you'd need to make a `GET` request for /management/ health to obtain the application's health information.

Whether or not you decide to change the Actuator base path, all Actuator endpoints in this chapter will be referred to without the base path for the sake of brevity. For example, when the /health endpoint is mentioned, it is the /{base path}/health endpoint that is intended, or more precisely, the /actuator/health endpoint if the base path hasn't been changed.

5.1.2 Enabling and disabling Actuator endpoints

You may have noticed that only the /health endpoint is enabled by default. Most Actuator endpoints carry sensitive information and should be secured. You can use Spring Security to lock down Actuator, but because Actuator isn't secured on its own, most of the endpoints are disabled by default, requiring you to opt in for the endpoints you wish to expose.

Two configuration properties, `management.endpoints.web.exposure.include` and `management.endpoints.web.exposure.exclude`, can be used to control which endpoints are exposed. Use `management.endpoints.web.exposure.include` to specify which endpoints you want to expose. For example, if you wish to expose only the /health, /info, /beans, and /conditions endpoints, you can specify that with the following configuration:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,beans,conditions
```

The `management.endpoints.web.exposure.include` property also accepts an asterisk (*) as a wildcard to indicate that all Actuator endpoints should be exposed, as shown here:

```
management:
  endpoints:
    web:
      exposure:
        include: '*'
```

If you want to expose all but a few endpoints, it's typically easier to include them all with a wildcard and then explicitly exclude a few. For example, to expose all Actuator endpoints except for /threaddump and /heapdump, you could set both the `management.endpoints.web.exposure.include` and `management.endpoints.web.exposure.exclude` properties like this:

```
management:
  endpoints:
    web:
      exposure:
        include: '*'
        exclude: threaddump,heapdump
```

Should you decide to expose more than /health and /info, it's probably a good idea to configure Spring Security to restrict access to the other endpoints. We'll look at how to secure Actuator endpoints in section 15.4. For now, though, let's look at how you can consume the HTTP endpoints exposed by Actuator.

15.2 Consuming Actuator endpoints

Actuator can bestow a veritable treasure trove of interesting and useful information about a running application by way of the HTTP endpoints listed in table 15.1. As HTTP endpoints, these can be consumed like any REST API, using whatever HTTP client you wish, including Spring's `RestTemplate` and `WebClient`, from a JavaScript application, or simply with the `curl` command-line client.

For the sake of exploring Actuator endpoints, we'll use the `curl` command-line client in this chapter. In chapter 16, I'll introduce you to Spring Boot Admin, which layers a user-friendly web application on top of an application's Actuator endpoints.

To get some idea of what endpoints Actuator has to offer, a `GET` request to Actuator's base path will provide HATEOAS links for each of the endpoints. Using `curl` to make a request to /actuator, you might get a response something like this (abridged to save space):

```
$ curl localhost:8080/actuator
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "auditevents": {
      "href": "http://localhost:8080/actuator/auditevents",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8080/actuator/beans",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    ...
  }
}
```

Because different libraries may contribute additional Actuator endpoints of their own, and because some endpoints may be not be exported, the actual results may vary from application to application.

In any event, the set of links returned from Actuator's base path serve as a map to all that Actuator has to offer. Let's begin our exploration of the Actuator landscape with the two endpoints that provide essential information about an application: the /health and /info endpoints.

5.2.1 Fetching essential application information

At the beginning of a typical visit to the doctor, we're usually asked two very basic questions: who are you and how do you feel? Although the words chosen by the doctor or nurse may be different, they ultimately want to know a little bit about the person they're treating and why you're seeing them.

Those same essential questions are what Actuator's /info and /health endpoints answer for a Spring Boot application. The /info endpoint tells you a little about the application, and the /health endpoint tells you how healthy the application is.

ASKING FOR INFORMATION ABOUT AN APPLICATION

To learn a little bit of information about a running Spring Boot application, you can ask the /info endpoint. By default, however, the /info endpoint isn't very informative. Here's what you might see when you make a request for it using `curl`:

```
$ curl localhost:8080/actuator/info
{}
```

Although it may seem that the /info endpoint isn't very useful, it's best to think of it as a clean canvas on which you may paint any information you'd like to present.

We have several ways to supply information for the /info endpoint to return, but the most straightforward way is to create one or more configuration properties where the property name is prefixed with `info.` For example, suppose that you want the response from the /info endpoint to include support contact information, including an email address and phone number. To do that, you can configure the following properties in the application.yml file:

```
info:
  contact:
    email: support@tacocloud.com
    phone: 822-625-6831
```

Neither the `info.contact.email` property nor the `info.contact.phone` property has any special meaning to Spring Boot or any bean that may be in the application context. However, by virtue of the fact that it's prefixed with *info*, the /info endpoint will now echo the value of the property in its response as follows:

```
{
  "contact": {
    "email": "support@tacocloud.com",
    "phone": "822-625-6831"
  }
}
```

In section 15.3.1, we'll look at a few other ways to populate the /info endpoint with useful information about an application.

INSPECTING APPLICATION HEALTH

Issuing an HTTP `GET` request for the /health endpoint results in a simple JSON response with the health status of your application. For example, here's what you might see when using `curl` to fetch the /health endpoint:

```
$ curl localhost:8080/actuator/health
{"status":"UP"}
```

You may be wondering how useful it is to have an endpoint that reports that the application is `UP`. What would it report if the application were down?

As it turns out, the status shown here is an aggregate status of one or more health indicators. Health indicators report the health of external systems that the application interacts with, such as databases, message brokers, and even Spring Cloud components such as Eureka and the Config Server. The health status of each indicator could be one of the following:

- `UP` —The external system is up and is reachable.
- `DOWN` —The external system is down or unreachable.
- `UNKNOWN` —The status of the external system is unclear.
- `OUT_OF_SERVICE` —The external system is reachable but is currently unavailable.

The health statuses of all health indicators are then aggregated into the application's overall health status, applying the following rules:

- If all health indicators are `UP`, then the application health status is `UP`.
- If one or more health indicators are `DOWN`, then the application health status is `DOWN`.

- If one or more health indicators are `OUT_OF_SERVICE`, then the application health status is `OUT_OF_SERVICE`.
- `UNKNOWN` health statuses are ignored and aren't rolled into the application's aggregate health.

By default, only the aggregate status is returned in response to a request for /health. You can configure the `management.endpoint.health.show-details` property, however, to show the full details of all health indicators, as shown next:

```
management:
  endpoint:
    health:
      show-details: always
```

The `management.endpoint.health.show-details` property defaults to `never`, but it can also be set to `always` to always show the full details of all health indicators, or to `when-authorized` to show the full details only when the requesting client is fully authorized.

Now when you issue a `GET` request to the /health endpoint, you get full health indicator details. Here's a sample of what that might look like for a service that integrates with the Mongo document database:

```
{
  "status": "UP",
  "details": {
    "mongo": {
      "status": "UP",
      "details": {
        "version": "3.5.5"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 499963170816,
        "free": 177284784128,
        "threshold": 10485760
      }
    }
  }
}
```

All applications, regardless of any other external dependencies, will have a health indicator for the filesystem named `diskSpace`. The `diskSpace` health indicator indicates the health of the filesystem (hopefully, `UP`), which is determined by how much free space is remaining. If the available disk space drops below the threshold, it will report a status of `DOWN`.

In the preceding example, there's also a `mongo` health indicator, which reports the status of the Mongo database. Details shown include the Mongo database version.

Autoconfiguration ensures that only health indicators that are pertinent to an application will appear in the response from the /health endpoint. In addition to the `mongo` and `diskSpace` health indicators, Spring Boot also provides health indicators for several other external databases and systems, including the following:

- Cassandra
- Config Server
- Couchbase
- Eureka
- Hystrix
- JDBC data sources
- Elasticsearch
- InfluxDB

- JMS message brokers
- LDAP
- Email servers
- Neo4j
- Rabbit message brokers
- Redis
- Solr

Additionally, third-party libraries may contribute their own health indicators. We'll look at how to write a custom health indicator in section 15.3.2.

As you've seen, the /health and /info endpoints provide general information about the running application. Meanwhile, other Actuator endpoints provide insight into the application configuration. Let's look at how Actuator can show how an application is configured.

15.2.2 Viewing configuration details

Beyond receiving general information about an application, it can be enlightening to understand how an application is configured. What beans are in the application context? What autoconfiguration conditions passed or failed? What environment properties are available to the application? How are HTTP requests mapped to controllers? What logging level are one or more packages or classes set to?

These questions are answered by Actuator's /beans, /conditions, /env, /configprops, /mappings, and /loggers endpoints. And in some cases, such as /env and /loggers, you can even adjust the configuration of a running application on the fly. We'll look at how each of these endpoints gives insight into the configuration of a running application, starting with the /beans endpoint.

GETTING A BEAN WIRING REPORT

The most essential endpoint for exploring the Spring application context is the /beans endpoint. This endpoint returns a JSON document describing every single bean in the application context, its Java type, and any of the other beans it's injected with.

A complete response from a `GET` request to /beans could easily fill this entire chapter. Instead of examining the complete response from /beans, let's consider the following snippet, which focuses on a single bean entry:

```
{
  "contexts": {
    "application-1": {
      "beans": {
...
        "ingredientsController": {
          "aliases": [],
          "scope": "singleton",
          "type": "tacos.ingredients.IngredientsController",
          "resource": "file [/Users/habuma/Documents/Workspaces/
            TacoCloud/ingredient-service/target/classes/tacos/
            ingredients/IngredientsController.class]",
          "dependencies": [
            "ingredientRepository"
          ]
        },
...
      },
      "parentId": null
    }
  }
}
```

At the root of the response is the `contexts` element, which includes one subelement for each Spring application context in the application. Within each application context is a `beans` element that holds details for all the beans in the application context.

In the preceding example, the bean shown is the one whose name is `ingredientsController`. You can see that it has no aliases, is scoped as a singleton, and is of type `tacos.ingredients.IngredientsController`. Moreover, the `resource` property gives the path to the class file that defines the bean. And the `dependencies` property lists all other beans that are injected into the given bean. In this case, the `ingredientsController` bean is injected with a bean whose name is `ingredientRepository`.

EXPLAINING AUTOCONFIGURATION

As you've seen, autoconfiguration is one of the most powerful things that Spring Boot offers. Sometimes, however, you may wonder why something has been autoconfigured. Or you may expect something to have been autoconfigured and are left wondering why it hasn't been. In that case, you can make a `GET` request to /conditions to get an explanation of what took place in autoconfiguration.

The autoconfiguration report returned from /conditions is divided into three parts: positive matches (conditional configuration that passed), negative matches (conditional configuration that failed), and unconditional classes. The following snippet from the response to a request to /conditions shows an example of each section:

```
{
  "contexts": {
    "application-1": {
      "positiveMatches": {
...
        "MongoDataAutoConfiguration#mongoTemplate": [
          {
            "condition": "OnBeanCondition",
            "message": "@ConditionalOnMissingBean (types:
               org.springframework.data.mongodb.core.MongoTemplate;
               SearchStrategy: all) did not find any beans"
          }
        ],
...
      },
      "negativeMatches": {
...
        "DispatcherServletAutoConfiguration": {
          "notMatched": [
            {
              "condition": "OnClassCondition",
              "message": "@ConditionalOnClass did not find required
                 class 'org.springframework.web.servlet.
                                      DispatcherServlet'"
            }
          ],
          "matched": []
        },
...
      },
      "unconditionalClasses": [
...
        "org.springframework.boot.autoconfigure.context.
                    ConfigurationPropertiesAutoConfiguration",
...
      ]
    }
  }
}
```

Under the `positiveMatches` section, you see that a `MongoTemplate` bean was configured by autoconfiguration because one didn't already exist. The autoconfiguration that caused this includes a `@ConditionalOnMissingBean` annotation, which passes off the bean to be configured if it hasn't already been explicitly configured. But in this case, no beans of type `MongoTemplate` were found, so autoconfiguration stepped in and configured one.

Under `negativeMatches`, Spring Boot autoconfiguration considered configuring a `DispatcherServlet`. But the `@ConditionalOnClass` conditional annotation failed because `DispatcherServlet` couldn't be found.

Finally, a `ConfigurationPropertiesAutoConfiguration` bean was configured unconditionally, as seen under the `unconditional-Classes` section. Configuration properties are foundational to how Spring Boot operates, so you should autoconfigure any configuration pertaining to configuration properties without any conditions.

INSPECTING THE ENVIRONMENT AND CONFIGURATION PROPERTIES

In addition to knowing how your application beans are wired together, you might also be interested in learning what environment properties are available and what configuration properties were injected into the beans.

When you issue a `GET` request to the /env endpoint, you'll receive a rather lengthy response that includes properties from all property sources in play in the Spring application. This includes properties from environment variables, JVM system properties, application.properties and application.yml files, and even the Spring Cloud Config Server (if the application is a client of the Config Server).

The following listing shows a greatly abridged example of the kind of response you might get from the /env endpoint, to give you some idea of the kind of information it provides.

Listing 15.1 The results from the /env endpoint

```
$ curl localhost:8080/actuator/env
{
  "activeProfiles": [
    "development"
  ],
  "propertySources": [
...
    {
      "name": "systemEnvironment",
      "properties": {
        "PATH": {
          "value": "/usr/bin:/bin:/usr/sbin:/sbin",
          "origin": "System Environment Property \"PATH\""
        },
...
        "HOME": {
          "value": "/Users/habuma",
          "origin": "System Environment Property \"HOME\""
        }
      }
    },
    {
      "name": "applicationConfig: [classpath:/application.yml]",
      "properties": {
        "spring.application.name": {
          "value": "ingredient-service",
          "origin": "class path resource [application.yml]:3:11"
        },
        "server.port": {
          "value": 8080,
          "origin": "class path resource [application.yml]:9:9"
        },
...
      }
```

```
    },
  ...
   ]
  }
```

Although the full response from /env provides even more information, what's shown in listing 15.1 contains a few noteworthy elements. First, notice that near the top of the response is a field named `activeProfiles`. In this case, it indicates that the `development` profile is active. If any other profiles were active, those would be listed as well.

Next, the `propertySources` field is an array containing an entry for every property source in the Spring application environment. In listing 15.1, only the `systemEnvironment` and an `applicationConfig` property source referencing the application.yml file are shown.

Within each property source is a listing of all properties provided by that source, paired with their values. In the case of the application.yml property source, the `origin` field for each property tells exactly where the property is set, including the line and column within application.yml.

The /env endpoint can also be used to fetch a specific property when that property's name is given as the second element of the path. For example, to examine the `server.port` property, submit a `GET` request for /env/server.port, as shown here:

```
$ curl localhost:8080/actuator/env/server.port
{
  "property": {
    "source": "systemEnvironment", "value": "8080"
  },
  "activeProfiles": [ "development" ],
  "propertySources": [
    { "name": "server.ports" },
    { "name": "mongo.ports" },
    { "name": "systemProperties" },
    { "name": "systemEnvironment",
      "property": {
        "value": "8080",
        "origin": "System Environment Property \"SERVER_PORT\""
      }
    },
    { "name": "random" },
    { "name": "applicationConfig: [classpath:/application.yml]",
      "property": {
        "value": 0,
        "origin": "class path resource [application.yml]:9:9"
      }
    },
    { "name": "springCloudClientHostInfo" },
    { "name": "refresh" },
    { "name": "defaultProperties" },
    { "name": "Management Server" }
  ]
}
```

As you can see, all property sources are still represented, but only those that set the specified property will contain any additional information. In this case, both the `systemEnvironment` property source and the application.yml property source had values for the `server.port` property. Because the `systemEnvironment` property source takes precedence over any of the property sources listed below it, its value of `8080` wins. The winning value is reflected near the top under the `property` field.

The /env endpoint can be used for more than just reading property values. By submitting a `POST` request to the /env endpoint, along with a JSON document with `name` and `value` fields, you can also set properties in the running application. For example, to set a property named `tacocloud.discount.code` to `TACOS1234`, you can use `curl` to submit the `POST` request at the command line like this:

```
$ curl localhost:8080/actuator/env \
    -d'{"name":"tacocloud.discount.code","value":"TACOS1234"}' \
    -H "Content-type: application/json"
{"tacocloud.discount.code":"TACOS1234"}
```

After submitting the property, the newly set property and its value are returned in the response. Later, should you decide you no longer need that property, you can submit a `DELETE` request to the /env endpoint as follows to delete all properties created through that endpoint:

```
$ curl localhost:8080/actuator/env -X DELETE
{"tacocloud.discount.code":"TACOS1234"}
```

As useful as setting properties through Actuator's API can be, it's important to be aware that any properties set with a `POST` request to the /env endpoint apply only to the application instance receiving the request, are temporary, and will be lost when the application restarts.

NAVIGATING HTTP REQUEST MAPPINGS

Although Spring MVC's (and Spring WebFlux's) programming model makes it easy to handle HTTP requests by simply annotating methods with request-mapping annotations, it can sometimes be challenging to get a big-picture understanding of all the kinds of HTTP requests that an application can handle and what kinds of components handle those requests.

Actuator's /mappings endpoint offers a one-stop view of every HTTP request handler in an application, whether it be from a Spring MVC controller or one of Actuator's own endpoints. To get a complete list of all the endpoints in a Spring Boot application, make a `GET` request to the /mappings endpoint, and you might receive something that's a little bit like the abridged response shown next.

Listing 15.2 HTTP mappings as shown by the /mappings endpoint

```
$ curl localhost:8080/actuator/mappings | jq
{
  "contexts": {
    "application-1": {
      "mappings": {
        "dispatcherHandlers": {
          "webHandler": [
...
            {
              "predicate": "{[/ingredients],methods=[GET]}",
              "handler": "public reactor.core.publisher.Flux<tacos.ingredients.Ingredient> tacos.ingredients.IngredientsController.allIngredients()",
              "details": {
                "handlerMethod": {
                  "className": "tacos.ingredients.IngredientsController",
                  "name": "allIngredients",
                  "descriptor": "()Lreactor/core/publisher/Flux;"
                },
                "handlerFunction": null,
                "requestMappingConditions": {
                  "consumes": [],
                  "headers": [],
                  "methods": [
                    "GET"
                  ],
                  "params": [],
                  "patterns": [
                    "/ingredients"
                  ],
                  "produces": []
                }
```

```
          }
        },
...
      ]
    }
   },
   "parentId": "application-1"
  },
  "bootstrap": {
   "mappings": {
    "dispatcherHandlers": {}
   },
   "parentId": null
  }
 }
}
```

Here, the response from the `curl` command line is piped to a utility called `jq` (**https://stedolan.github.io/jq/**), which, among other things, pretty-prints the JSON returned from the request in an easily readable format. For the sake of brevity, this response has been abridged to show only a single request handler. Specifically, it shows that `GET` requests for /ingredients will be handled by the `allIngredients()` method of `IngredientsController`.

MANAGING LOGGING LEVELS

Logging is an important feature of any application. Logging can provide a means of auditing as well as a crude means of debugging.

Setting logging levels can be quite a balancing act. If you set the logging level to be too verbose, there may be too much noise in the logs, and finding useful information may be difficult. On the other hand, if you set logging levels to be too slack, the logs may not be of much value in understanding what an application is doing.

Logging levels are typically applied on a package-by-package basis. If you're ever wondering what logging levels are set in your running Spring Boot application, you can issue a `GET` request to the /loggers endpoint. The following JSON code shows an excerpt from a response to /loggers:

```
{
 "levels": [ "OFF", "ERROR", "WARN", "INFO", "DEBUG", "TRACE" ],
 "loggers": {
  "ROOT": {
   "configuredLevel": "INFO", "effectiveLevel": "INFO"
  },
...
  "org.springframework.web": {
   "configuredLevel": null, "effectiveLevel": "INFO"
  },
...
  "tacos": {
   "configuredLevel": null, "effectiveLevel": "INFO"
  },
  "tacos.ingredients": {
   "configuredLevel": null, "effectiveLevel": "INFO"
  },
  "tacos.ingredients.IngredientServiceApplication": {
   "configuredLevel": null, "effectiveLevel": "INFO"
  }
 }
}
```

The response starts off with a list of all valid logging levels. After that, the `loggers` element lists logging-level details for each package in the application. The `configuredLevel` property shows the logging level that has been explicitly configured (or

`null` if it hasn't been explicitly configured). The `effectiveLevel` property gives the effective logging level, which may have been inherited from a parent package or from the root logger.

Although this excerpt shows logging levels only for the root logger and four packages, the complete response will include logging-level entries for every single package in the application, including those for libraries that are in use. If you'd rather focus your request on a specific package, you can specify the package name as an extra path component in the request.

For example, if you just want to know what logging levels are set for the `tacocloud .ingredients` package, you can make a request to /loggers/tacos.ingredients as follows:

```
{
  "configuredLevel": null,
  "effectiveLevel": "INFO"
}
```

Aside from returning the logging levels for the application packages, the /loggers endpoint also allows you to change the configured logging level by issuing a `POST` request. For example, suppose you want to set the logging level of the `tacocloud .ingredients` package to `DEBUG`. The following `curl` command will achieve that:

```
$ curl localhost:8080/actuator/loggers/tacos/ingredients \
    -d'{"configuredLevel":"DEBUG"}' \
    -H"Content-type: application/json"
```

Now that the logging level has been changed, you can issue a `GET` request to /loggers/ tacos/ingredients as shown here to see that it has been changed:

```
{
  "configuredLevel": "DEBUG",
  "effectiveLevel": "DEBUG"
}
```

Notice that where the `configuredLevel` was previously `null`, it's now `DEBUG`. That change carries over to the `effectiveLevel` as well. But what's most important is that if any code in that package logs anything at debug level, the log files will include that debug-level information.

5.2.3 Viewing application activity

It can be useful to keep an eye on activity in a running application, including the kinds of HTTP requests that the application is handling and the activity of all of the threads in the application. For this, Actuator provides the /httptrace, /threaddump, and /heapdump endpoints.

The /heapdump endpoint is perhaps the most difficult Actuator endpoint to describe in any detail. Put succinctly, it downloads a gzip-compressed HPROF heap dump file that can be used to track down memory or thread issues. For the sake of space and because use of the heap dump is a rather advanced feature, I'm going to limit coverage of the /heapdump endpoint to this paragraph.

TRACING HTTP ACTIVITY

The /httptrace endpoint reports details on the most recent 100 requests handled by an application. Details included are the request method and path, a timestamp indicating when the request was handled, headers from both the request and the response, and the time taken handling the request.

The following snippet of JSON code shows a single entry from the response of the /httptrace endpoint:

```
{
  "traces": [
    {
      "timestamp": "2020-06-03T23:41:24.494Z",
      "principal": null,
      "session": null,
      "request": {
        "method": "GET",
        "uri": "http://localhost:8080/ingredients",
        "headers": {
          "Host": ["localhost:8080"],
          "User-Agent": ["curl/7.54.0"],
          "Accept": ["*/*"]
        },
        "remoteAddress": null
      },
      "response": {
        "status": 200,
        "headers": {
          "Content-Type": ["application/json;charset=UTF-8"]
        }
      },
      "timeTaken": 4
    },
...
  ]
}
```


Although this information may be useful for debugging purposes, it's even more interesting when the trace data is tracked over time, providing insight into how busy the application was at any given time as well as how many requests were successful compared to how many failed, based on the value of the response status. In chapter 16, you'll see how Spring Boot Admin captures this information into a running graph that visualizes the HTTP trace information over a period of time.

MONITORING THREADS

In addition to HTTP request tracing, thread activity can also be useful in determining what's going on in a running application. The /threaddump endpoint produces a snapshot of current thread activity. The following snippet from a /threaddump response gives a taste of what this endpoint provides:


```
{
  "threadName": "reactor-http-nio-8",
  "threadId": 338,
  "blockedTime": -1,
  "blockedCount": 0,
  "waitedTime": -1,
  "waitedCount": 0,
  "lockName": null,
  "lockOwnerId": -1,
  "lockOwnerName": null,
  "inNative": true,
  "suspended": false,
  "threadState": "RUNNABLE",
  "stackTrace": [
    {
      "methodName": "kevent0",
      "fileName": "KQueueArrayWrapper.java",
      "lineNumber": -2,
      "className": "sun.nio.ch.KQueueArrayWrapper",
      "nativeMethod": true
    },
    {
```

```
      "methodName": "poll",
      "fileName": "KQueueArrayWrapper.java",
      "lineNumber": 198,
      "className": "sun.nio.ch.KQueueArrayWrapper",
      "nativeMethod": false
    },
...
  ],
  "lockedMonitors": [
    {
      "className": "io.netty.channel.nio.SelectedSelectionKeySet",
      "identityHashCode": 1039768944,
      "lockedStackDepth": 3,
      "lockedStackFrame": {
        "methodName": "lockAndDoSelect",
        "fileName": "SelectorImpl.java",
        "lineNumber": 86,
        "className": "sun.nio.ch.SelectorImpl",
        "nativeMethod": false
      }
    },
...
  ],
  "lockedSynchronizers": [],
  "lockInfo": null
}
```


The complete thread dump report includes every thread in the running application. To save space, the thread dump here shows an abridged entry for a single thread. As you can see, it includes details regarding the blocking and locking status of the thread, among other thread specifics. There's also a stack trace that gives some insight into which area of the code the thread is spending time on.

Because the /threaddump endpoint provides a snapshot of thread activity only at the time it was requested, it can be difficult to get a full picture of how threads are behaving over time. In chapter 16, you'll see how Spring Boot Admin can monitor the /threaddump endpoint in a live view.

6.2.4 Tapping runtime metrics

The /metrics endpoint can report many metrics produced by a running application, including memory, processor, garbage collection, and HTTP requests. Actuator provides more than two dozen categories of metrics out of the box, as evidenced by the following list of metrics categories returned when issuing a `GET` request to /metrics:

```
$ curl localhost:8080/actuator/metrics | jq
{
  "names": [
    "jvm.memory.max",
    "process.files.max",
    "jvm.gc.memory.promoted",
    "http.server.requests",
    "system.load.average.1m",
    "jvm.memory.used",
    "jvm.gc.max.data.size",
    "jvm.memory.committed",
    "system.cpu.count",
    "logback.events",
    "jvm.buffer.memory.used",
    "jvm.threads.daemon",
    "system.cpu.usage",
    "jvm.gc.memory.allocated",
    "jvm.threads.live",
    "jvm.threads.peak",
```

```
    "process.uptime",
    "process.cpu.usage",
    "jvm.classes.loaded",
    "jvm.gc.pause",
    "jvm.classes.unloaded",
    "jvm.gc.live.data.size",
    "process.files.open",
    "jvm.buffer.count",
    "jvm.buffer.total.capacity",
    "process.start.time"
  ]
}
```

So many metrics are covered that it would be impossible to discuss them all in any meaningful way in this chapter. Instead, let's focus on one category of metrics, `http.server.requests`, as an example of how to consume the /metrics endpoint.

If instead of simply requesting /metrics, you were to issue a `GET` request for /metrics/{metrics name}, you'd receive more detail about the metrics for that category. In the case of `http.server.requests`, a `GET` request for /metrics/ http.server .requests returns data that looks like the following:

```
$ curl localhost:8080/actuator/metrics/http.server.requests
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 2103 },
    { "statistic": "TOTAL_TIME", "value": 18.086334315 },
    { "statistic": "MAX", "value": 0.028926313 }
  ],
  "availableTags": [
    { "tag": "exception",
      "values": [ "ResponseStatusException",
            "IllegalArgumentException", "none" ] },
    { "tag": "method", "values": [ "GET" ] },
    { "tag": "uri",
      "values": [
        "/actuator/metrics/{requiredMetricName}",
        "/actuator/health", "/actuator/info", "/ingredients",
        "/actuator/metrics", "/**" ] },
    { "tag": "status", "values": [ "404", "500", "200" ] }
  ]
}
```

The most significant portion of this response is the `measurements` section, which includes all the metrics for the requested category. In this case, it reports that there have been 2,103 HTTP requests. The total time spent handling those requests is 18.086334315 seconds, and the maximum time spent processing any request is 0.028926313 seconds.

Those generic metrics are interesting, but you can narrow down the results further by using the tags listed under `available-Tags`. For example, you know that there have been 2,103 requests, but what's unknown is how many of them resulted in an HTTP 200 versus an HTTP 404 or HTTP 500 response `status`. Using the status tag, you can get metrics for all requests resulting in an HTTP 404 status like this:

```
$ curl localhost:8080/actuator/metrics/http.server.requests? \
                          tag=status:404
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 31 },
    { "statistic": "TOTAL_TIME", "value": 0.522061212 },
    { "statistic": "MAX", "value": 0 }
  ],
```

```
  "availableTags": [
   { "tag": "exception",
     "values": [ "ResponseStatusException", "none" ] },
   { "tag": "method", "values": [ "GET" ] },
   { "tag": "uri",
     "values": [
       "/actuator/metrics/{requiredMetricName}", "/**" ] }
  ]
}
```

By specifying the tag name and value with the `tag` request attribute, you now see metrics specifically for requests that resulted in an HTTP 404 response. This shows that there were 31 requests resulting in a 404, and it took 0.522061212 seconds to serve them all. Moreover, it's clear that some of the failing requests were `GET` requests for /actuator/metrics/{requiredMetricsName} (although it's unclear what the {requiredMetricsName} path variable resolved to). And some were for some other path, captured by the `/**` wildcard path.

Hmmm . . . what if you want to know how many of those HTTP 404 responses were for the /** path? All you need to do to filter this further is to specify the `uri` tag in the request, like this:

```
% curl "localhost:8080/actuator/metrics/http.server.requests? \
                        tag=status:404&tag=uri:/**"
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 30 },
    { "statistic": "TOTAL_TIME", "value": 0.519791548 },
    { "statistic": "MAX", "value": 0 }
  ],
  "availableTags": [
    { "tag": "exception", "values": [ "ResponseStatusException" ] },
    { "tag": "method", "values": [ "GET" ] }
  ]
}
```

Now you can see that there were 30 requests for some path that matched /** that resulted in an HTTP 404 response, and it took a total of 0.519791548 seconds to handle those requests.

You'll also notice that as you refine the request, the available tags are more limited. The tags offered are only those that match the requests captured by the displayed metrics. In this case, the `exception` and `method` tags each have only a single value; it's obvious that all 30 of the requests were `GET` requests that resulted in a 404 because of a `ResponseStatusException`.

Navigating the /metrics endpoint can be a tricky business, but with a little practice, it's not impossible to get the data you're looking for. In chapter 16, you'll see how Spring Boot Admin makes consuming data from the /metrics endpoint much easier.

Although the information presented by Actuator endpoints offers useful insight into the inner workings of a running Spring Boot application, it's not well suited for human consumption. Because Actuator endpoints are REST endpoints, the data they provide is intended for consumption by some other application, perhaps a UI. With that in mind, let's see how you can present Actuator information in a user-friendly web application.

15.3 Customizing Actuator

One of the greatest features of Actuator is that it can be customized to meet the specific needs of an application. A few of the endpoints themselves allow for customization. Meanwhile, Actuator itself allows you to create custom endpoints.

Let's look at a few ways that Actuator can be customized, starting with ways to add information to the /info endpoint.

15.3.1 Contributing information to the /info endpoint

As you saw in section 15.2.1, the /info endpoint starts off empty and uninformative. But you can easily add data to it by creating properties that are prefixed with `info.`

Although prefixing properties with `info.` is a very easy way to get custom data into the /info endpoint, it's not the only way. Spring Boot offers an interface named `InfoContributor` that allows you to programmatically add any information you want to the /info endpoint response. Spring Boot even comes ready with a couple of useful implementations of `InfoContributor` that you'll no doubt find useful.

Let's see how you can write your own `InfoContributor` to add some custom info to the /info endpoint.

CREATING A CUSTOM INFOCONTRIBUTOR

Suppose you want to add some simple statistics regarding Taco Cloud to the /info endpoint. For example, let's say you want to include information about how many tacos have been created. To do that, you can write a class that implements `InfoContributor`, inject it with `TacoRepository`, and then publish whatever count that `TacoRepository` gives you as information to the /info endpoint. The next listing shows how you might implement such a contributor.

Listing 15.3 A custom implementation of `InfoContributor`

```
package tacos.actuator;

import java.util.HashMap;
import java.util.Map;

import org.springframework.boot.actuate.info.Info.Builder;
import org.springframework.boot.actuate.info.InfoContributor;
import org.springframework.stereotype.Component;

import tacos.data.TacoRepository;

@Component
public class TacoCountInfoContributor implements InfoContributor {
 private TacoRepository tacoRepo;

 public TacoCountInfoContributor(TacoRepository tacoRepo) {
  this.tacoRepo = tacoRepo;
 }

 @Override
 public void contribute(Builder builder) {
  long tacoCount = tacoRepo.count().block();
  Map<String, Object> tacoMap = new HashMap<String, Object>();
  tacoMap.put("count", tacoCount);
  builder.withDetail("taco-stats", tacoMap);
 }
}
```

By implementing `InfoContributor`, `TacoCountInfoContributor` is required to implement the `contribute()` method. This method is given a `Builder` object on which the `contribute()` method makes a call to `withDetail()` to add info details. In your implementation, you consult `TacoRepository` by calling its `count()` method to find out how many tacos have been created. In this particular case, you're working with a reactive repository, so you need to call `block()` to get the count out of the returned `Mono<Long>`. Then you put that count into a `Map`, which you then give to the builder with the label `taco-stats`. The results of the /info endpoint will include that count, as shown here:

```
{
 "taco-stats": {
  "count": 44
```

```
  }
}
```

As you can see, an implementation of `InfoContributor` is able to use whatever means necessary to contribute information. This is in contrast to simply prefixing a property with `info.`, which, although simple, is limited to static values.

INJECTING BUILD INFORMATION INTO THE /INFO ENDPOINT

Spring Boot comes with a few built-in implementations of `InfoContributor` that automatically add information to the results of the /info endpoint. Among them is `BuildInfoContributor`, which adds information from the project build file into the /info endpoint results. The basic information includes the project version, the timestamp of the build, and the host and user who performed the build.

To enable build information to be included in the results of the /info endpoint, add the `build-info` goal to the Spring Boot Maven Plugin executions, as follows:

```
<build>
 <plugins>
  <plugin>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-maven-plugin</artifactId>
   <executions>
    <execution>
     <goals>
      <goal>build-info</goal>
     </goals>
    </execution>
   </executions>
  </plugin>
 </plugins>
</build>
```

If you're using Gradle to build your project, you can simply add the following lines to your build.gradle file:

```
springBoot {
 buildInfo()
}
```

In either event, the build will produce a file named build-info.properties in the distributable JAR or WAR file that `BuildInfoContributor` will consume and contribute to the /info endpoint. The following snippet from the /info endpoint response shows the build information that's contributed:

```
{
 "build": {
  "artifact": "tacocloud",
  "name": "taco-cloud",
  "time": "2021-08-08T23:55:16.379Z",
  "version": "0.0.15-SNAPSHOT",
  "group": "sia"
 },
}
```

This information is useful for understanding exactly which version of an application is running and when it was built. By performing a `GET` request to the /info endpoint, you'll know whether you're running the latest and greatest build of the project.

EXPOSING GIT COMMIT INFORMATION

Assuming that your project is kept in Git for source code control, you may want to include Git commit information in the /info endpoint. To do that, you'll need to add the following plugin in the Maven project pom.xml:

```
<build>
 <plugins>
...
  <plugin>
   <groupId>pl.project13.maven</groupId>
   <artifactId>git-commit-id-plugin</artifactId>
  </plugin>
 </plugins>
</build>
```

If you're a Gradle user, don't worry. There's an equivalent plugin for you to add to your build.gradle file, shown here:

```
plugins {
 id "com.gorylenko.gradle-git-properties" version "2.3.1"
}
```

Both of these plugins do essentially the same thing: they generate a build-time artifact named git.properties that contains all of the Git metadata for the project. A special `InfoContributor` implementation discovers that file at runtime and exposes its contents as part of the /info endpoint.

Of course, to generate the git.properties file, the project needs to have Git commit metadata. That is, it must be a clone of a Git repository or be a newly initialized local Git repository with at least one commit. If not, then either of these plugins will fail. You can, however, configure them to ignore the missing Git metadata. For the Maven plugin, set the `failOnNoGitDirectory` property to `false` like this:

```
<build>
 <plugins>
...
  <plugin>
   <groupId>pl.project13.maven</groupId>
   <artifactId>git-commit-id-plugin</artifactId>
   <configuration>
    <failOnNoGitDirectory>false</failOnNoGitDirectory>
   </configuration>
  </plugin>
 </plugins>
</build>
```

Similarly, you can set the `failOnNoGitDirectory` property in Gradle by specifying it under `gitProperties` like this:

```
gitProperties {
 failOnNoGitDirectory = false
}
```

In its simplest form, the Git information presented in the /info endpoint includes the Git branch, commit hash, and timestamp that the application was built against, as shown here:

```
{
 "git": {
  "branch": "main",
  "commit": {
```

```
    "id": "df45505",
    "time": "2021-08-08T21:51:12Z"
   }
  },
 ...
}
```

This information is quite definitive in describing the state of the code when the project was built. But by setting the `management.info.git.mode` property to `full`, you can get extremely detailed information about the Git commit that was in play when the project was built, as shown in the next code sample:

```
management:
  info:
    git:
      mode: full
```

The following listing shows a sample of what the full Git info might look like.

Listing 15.4 Full Git commit info exposed through the /info endpoint

```
"git": {
  "local": {
    "branch": {
      "ahead": "8",
      "behind": "0"
    }
  },
  "commit": {
    "id": {
      "describe-short": "df45505-dirty",
      "abbrev": "df45505",
      "full": "df455055daaf3b1347b0ad1d9dca4ebbc6067810",
      "describe": "df45505-dirty"
    },
    "message": {
      "short": "Apply chapter 18 edits",
      "full": "Apply chapter 18 edits"
    },
    "user": {
      "name": "Craig Walls",
      "email": "craig@habuma.com"
    },
    "author": {
      "time": "2021-08-08T15:51:12-0600"
    },
    "committer": {
      "time": "2021-08-08T15:51:12-0600"
    },
    "time": "2021-08-08T21:51:12Z"
  },
  "branch": "master",
  "build": {
    "time": "2021-08-09T00:13:37Z",
    "version": "0.0.15-SNAPSHOT",
    "host": "Craigs-MacBook-Pro.local",
    "user": {
      "name": "Craig Walls",
      "email": "craig@habuma.com"
    }
  },
  "tags": "",
```

```
  "total": {
    "commit": {
      "count": "196"
    }
  },
  "closest": {
    "tag": {
      "commit": {
        "count": ""
      },
      "name": ""
    }
  },
  "remote": {
    "origin": {
      "url": "git@github.com:habuma/spring-in-action-6-samples.git"
    }
  },
  "dirty": "true"
},
```

In addition to the timestamp and abbreviated Git commit hash, the full version includes the name and email of the user who committed the code as well as the commit message and other information, allowing you to pinpoint exactly what code was used to build the project. In fact, notice that the `dirty` field in listing 15.4 is `true`, indicating that some uncommitted changes existed in the build directory when the project was built. It doesn't get much more definitive than that!

15.3.2 Defining custom health indicators

Spring Boot comes with several out-of-the-box health indicators that provide health information for many common external systems that a Spring application may integrate with. But at some point, you may find that you're interacting with some external system that Spring Boot neither anticipated nor provided a health indicator for.

For instance, your application may integrate with a legacy mainframe application, and the health of your application may be affected by the health of the legacy system. To create a custom health indicator, all you need to do is create a bean that implements the `HealthIndicator` interface.

As it turns out, the Taco Cloud services have no need for a custom health indicator, because the ones provided by Spring Boot are more than sufficient. But to demonstrate how you can develop a custom health indicator, consider the next listing, which shows a simple implementation of `HealthIndicator` in which health is determined somewhat randomly by the time of day.

Listing 15.5 An unusual implementation of `HealthIndicator`

```
package tacos.actuator;

import java.util.Calendar;
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;


@Component
public class WackoHealthIndicator
      implements HealthIndicator {
  @Override
  public Health health() {
    int hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY);
    if (hour > 12) {
      return Health
          .outOfService()
          .withDetail("reason",
```

```
            "I'm out of service after lunchtime")
        .withDetail("hour", hour)
        .build();
    }

    if (Math.random() <= 0.1) {
      return Health
        .down()
        .withDetail("reason", "I break 10% of the time")
        .build();
    }
    return Health
      .up()
      .withDetail("reason", "All is good!")
      .build();
  }
}
```

This crazy health indicator first checks what the current time is, and if it's after noon, returns a health status of `OUT_OF_SERVICE`, with a few details explaining the reason for that status. Even if it's before lunch, there's a 10% chance that the health indicator will report a `DOWN` status, because it uses a random number to decide whether or not it's up. If the random number is less than 0.1, the status will be reported as `DOWN`. Otherwise, the status will be `UP`.

Obviously, the health indicator in listing 15.5 isn't going to be very useful in any real-world applications. But imagine that instead of consulting the current time or a random number, it were to make a remote call to some external system and determine the status based on the response it receives. In that case, it would be a very useful health indicator.

15.3.3 Registering custom metrics

In section 15.2.4, we looked at how you could navigate the /metrics endpoint to consume various metrics published by Actuator, with a focus on metrics pertaining to HTTP requests. The metrics provided by Actuator are very useful, but the /metrics endpoint isn't limited to only those built-in metrics.

Ultimately, Actuator metrics are implemented by Micrometer (**https://micrometer.io/**), a vendor-neutral metrics facade that makes it possible for applications to publish any metrics they want and to display them in the third-party monitoring system of their choice, including support for Prometheus, Datadog, and New Relic, among others.

The most basic means of publishing metrics with Micrometer is through Micrometer's `MeterRegistry`. In a Spring Boot application, all you need to do to publish metrics is inject a `MeterRegistry` wherever you may need to publish counters, timers, or gauges that capture the metrics for your application.

As an example of publishing custom metrics, suppose you want to keep counters for the numbers of tacos that have been created with different ingredients. That is, you want to track how many tacos have been made with lettuce, ground beef, flour tortillas, or any of the available ingredients. The `TacoMetrics` bean in the next listing shows how you might use `MeterRegistry` to gather that information.

Listing 15.6 `TacoMetrics` registers metrics around taco ingredients

```
package tacos.actuator;

import java.util.List;
import org.springframework.data.rest.core.event.AbstractRepositoryEventListener;
import org.springframework.stereotype.Component;
import io.micrometer.core.instrument.MeterRegistry;
import tacos.Ingredient;
import tacos.Taco;

@Component
public class TacoMetrics extends AbstractRepositoryEventListener<Taco> {
```

```
  private MeterRegistry meterRegistry;

  public TacoMetrics(MeterRegistry meterRegistry) {
    this.meterRegistry = meterRegistry;
  }

  @Override
  protected void onAfterCreate(Taco taco) {
    List<Ingredient> ingredients = taco.getIngredients();
    for (Ingredient ingredient : ingredients) {
      meterRegistry.counter("tacocloud",
          "ingredient", ingredient.getId()).increment();
    }
  }
}
```

As you can see, `TacoMetrics` is injected through its constructor with a `MeterRegistry`. It also extends `AbstractRepositoryEventListener`, a Spring Data class that enables the interception of repository events and overrides the `onAfterCreate()` method so that it can be notified any time a new `Taco` object is saved.

Within `onAfterCreate()`, a counter is declared for each `ingredient` where the tag name is ingredient and the tag value is equal to the ingredient ID. If a counter with that tag already exists, it will be reused. The counter is incremented, indicating that another taco has been created for the ingredient.

After a few tacos have been created, you can start querying the /metrics endpoint for ingredient counts. A `GET` request to /metrics/tacocloud yields some unfiltered metric counts, as shown next:

```
$ curl localhost:8080/actuator/metrics/tacocloud
{
  "name": "tacocloud",
  "measurements": [ { "statistic": "COUNT", "value": 84 }
  ],
  "availableTags": [
    {
      "tag": "ingredient",
      "values": [ "FLTO", "CHED", "LETC", "GRBF",
                  "COTO", "JACK", "TMTO", "SLSA"]
    }
  ]
}
```

The count value under `measurements` doesn't mean much here, because it's a sum of all the counts for all ingredients. But let's suppose you want to know how many tacos have been created with flour tortillas (`FLTO`). All you need to do is specify the `ingredient` tag with a value of `FLTO` as follows:

```
$ curl localhost:8080/actuator/metrics/tacocloud?tag=ingredient:FLTO

{
  "name": "tacocloud",
  "measurements": [
    { "statistic": "COUNT", "value": 39 }
  ],
  "availableTags": []
}
```

Now it's clear that 39 tacos have had flour tortillas as one of their ingredients.

15.3.4 Creating custom endpoints

At first glance, you might think that Actuator's endpoints are implemented as nothing more than Spring MVC controllers. But as you'll see in chapter 17, the endpoints are also exposed as JMX MBeans as well as through HTTP requests. Therefore, there must be something more to these endpoints than just a controller class.

In fact, Actuator endpoints are defined quite differently from controllers. Instead of a class that's annotated with `@Controller` or `@RestController`, Actuator endpoints are defined with classes that are annotated with `@Endpoint`.

What's more, instead of using HTTP-named annotations such as `@GetMapping`, `@PostMapping`, or `@DeleteMapping`, Actuator endpoint operations are defined by methods annotated with `@ReadOperation`, `@WriteOperation`, and `@DeleteOperation`. These annotations don't imply any specific communication mechanism and, in fact, allow Actuator to communicate by any variety of communication mechanisms, HTTP, and JMX out of the box. To demonstrate how to write a custom Actuator endpoint, consider `NotesEndpoint` in the next listing.

Listing 15.7 A custom endpoint for taking notes

```
package tacos.actuator;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import org.springframework.boot.actuate.endpoint.annotation.DeleteOperation;
import org.springframework.boot.actuate.endpoint.annotation.Endpoint;
import org.springframework.boot.actuate.endpoint.annotation.ReadOperation;
import org.springframework.boot.actuate.endpoint.annotation.WriteOperation;
import org.springframework.stereotype.Component;

@Component
@Endpoint(id="notes", enableByDefault=true)
public class NotesEndpoint {

 private List<Note> notes = new ArrayList<>();

 @ReadOperation
 public List<Note> notes() {
  return notes;
 }

 @WriteOperation
 public List<Note> addNote(String text) {
  notes.add(new Note(text));
  return notes;
 }

 @DeleteOperation
 public List<Note> deleteNote(int index) {
  if (index < notes.size()) {
   notes.remove(index);
  }
  return notes;
 }

 class Note {
  private Date time = new Date();
  private final String text;

  public Note(String text) {
   this.text = text;
  }

  public Date getTime() {
```

```
      return time;
    }

    public String getText() {
      return text;
    }
  }
}
```

This endpoint is a simple note-taking endpoint, wherein one can submit a note with a write operation, read the list of notes with a read operation, and remove a note with the delete operation. Admittedly, this endpoint isn't very useful as far as Actuator endpoints go. But when you consider that the out-of-the-box Actuator endpoints cover so much ground, it's difficult to come up with a practical example of a custom Actuator endpoint.

At any rate, the `NotesEndpoint` class is annotated with `@Component` so that it will be picked up by Spring's component scanning and instantiated as a bean in the Spring application context. But more relevant to this discussion, it's also annotated with `@Endpoint`, making it an Actuator endpoint with an ID of `notes`. And it's enabled by default so that you won't need to explicitly enable it by including it in the `management.web.endpoints.web.exposure.include` configuration property.

As you can see, `NotesEndpoint` offers one of each kind of operation:

- The `notes()` method is annotated with `@ReadOperation`. When invoked, it will return a list of available notes. In HTTP terms, this means it will handle an HTTP `GET` request for /actuator/notes and respond with a JSON list of notes.
- The `addNote()` method is annotated with `@WriteOperation`. When invoked, it will create a new note from the given text and add it to the list. In HTTP terms, it handles a `POST` request where the body of the request is a JSON object with a `text` property. It finishes by responding with the current state of the notes list.
- The `deleteNote()` method is annotated with `@DeleteOperation`. When invoked, it will delete the note at the given index. In HTTP terms, this endpoint handles `DELETE` requests where the index is given as a request parameter.

To see this in action, you can use `curl` to poke about with this new endpoint. First, add a couple of notes, using two separate `POST` requests, as shown here:

```
$ curl localhost:8080/actuator/notes \
        -d'{"text":"Bring home milk"}' \
        -H"Content-type: application/json"
[{"time":"2020-06-08T13:50:45.085+0000","text":"Bring home milk"}]

$ curl localhost:8080/actuator/notes \
        -d'{"text":"Take dry cleaning"}' \
        -H"Content-type: application/json"
[{"time":"2021-07-03T12:39:13.058+0000","text":"Bring home milk"},
 {"time":"2021-07-03T12:39:16.012+0000","text":"Take dry cleaning"}]
```

As you can see, each time a new note is posted, the endpoint responds with the newly appended list of notes. But if later you want to view the list of notes, you can do a simple `GET` request like so:

```
$ curl localhost:8080/actuator/notes
[{"time":"2021-07-03T12:39:13.058+0000","text":"Bring home milk"},
 {"time":"2021-07-03T12:39:16.012+0000","text":"Take dry cleaning"}]
```

If you decide to remove one of the notes, a `DELETE` request with an `index` request parameter, shown next, should do the trick:

```
$ curl localhost:8080/actuator/notes?index=1 -X DELETE
[{"time":"2021-07-03T12:39:13.058+0000","text":"Bring home milk"}]
```

It's important to note that although I've shown only how to interact with the endpoint using HTTP, it will also be exposed as an MBean that can be accessed using whatever JMX client you choose. But if you want to limit it to only exposing an HTTP endpoint, you can annotate the endpoint class with `@WebEndpoint` instead of `@Endpoint` as follows:

```
@Component
@WebEndpoint(id="notes", enableByDefault=true)
public class NotesEndpoint {
  ...
}
```

Likewise, if you prefer an MBean-only endpoint, annotate the class with `@JmxEndpoint`.

15.4 Securing Actuator

The information presented by Actuator is probably not something that you would want prying eyes to see. Moreover, because Actuator provides a few operations that let you change environment properties and logging levels, it's probably a good idea to secure Actuator so that only clients with proper access will be allowed to consume its endpoints.

Even though it's important to secure Actuator, security is outside of Actuator's responsibilities. Instead, you'll need to use Spring Security to secure Actuator. And because Actuator endpoints are just paths in the application like any other path in the application, there's nothing unique about securing Actuator versus any other application path. Everything we discussed in chapter 5 applies when securing Actuator endpoints.

Because all Actuator endpoints are gathered under a common base path of /actuator (or possibly some other base path if the `management.endpoints.web.base-path` property is set), it's easy to apply authorization rules to all Actuator endpoints across the board. For example, to require that a user have `ROLE_ADMIN` authority to invoke Actuator endpoints, you might override the `configure()` method of `WebSecurityConfigurerAdapter` like this:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .authorizeRequests()
      .antMatchers("/actuator/**").hasRole("ADMIN")

    .and()

    .httpBasic();
}
```

This requires that all requests be from an authenticated user with `ROLE_ADMIN` authority. It also configures HTTP basic authentication so that client applications can submit encoded authentication information in their request `Authorization` headers.

The only real problem with securing Actuator this way is that the path to the endpoints is hardcoded as /actuator/**. If this were to change because of a change to the `management.endpoints.web.base-path` property, it would no longer work. To help with this, Spring Boot also provides `EndpointRequest`—a request matcher class that makes this even easier and less dependent on a given `String` path. Using `EndpointRequest`, you can apply the same security requirements for Actuator endpoints without hardcoding the /actuator/** path, as shown here:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .requestMatcher(EndpointRequest.toAnyEndpoint())
      .authorizeRequests()
        .anyRequest().hasRole("ADMIN")
    .and()
```

```
    .httpBasic();
}
```

The `EndpointRequest.toAnyEndpoint()` method returns a request matcher that matches any Actuator endpoint. If you'd like to exclude some of the endpoints from the request matcher, you can call `excluding()`, specifying them by name as follows:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .requestMatcher(
       EndpointRequest.toAnyEndpoint()
                  .excluding("health", "info"))
    .authorizeRequests()
     .anyRequest().hasRole("ADMIN")
  .and()
    .httpBasic();
}
```

On the other hand, should you wish to apply security to only a handful of Actuator endpoints, you can specify those endpoints by name by calling `to()` instead of `toAnyEndpoint()`, like this:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
  http
    .requestMatcher(EndpointRequest.to(
          "beans", "threaddump", "loggers"))
    .authorizeRequests()
     .anyRequest().hasRole("ADMIN")
  .and()
    .httpBasic();
}
```

This limits Actuator security to only the /beans, /threaddump, and /loggers endpoints. All other Actuator endpoints are left wide open.

Summary

- Spring Boot Actuator provides several endpoints, both as HTTP and JMX MBeans, that let you peek into the inner workings of a Spring Boot application.
- Most Actuator endpoints are disabled by default but can be selectively exposed by setting `management.endpoints.web.exposure.include` and `management.endpoints.web.exposure.exclude`.
- Some endpoints, such as the /loggers and /env endpoints, allow for write operations to change a running application's configuration on the fly.
- Details regarding an application's build and Git commit can be exposed in the /info endpoint.
- An application's health can be influenced by a custom health indicator, tracking the health of an externally integrated application.
- Custom application metrics can be registered through Micrometer, which affords Spring Boot applications instant integration with several popular metrics engines such as Datadog, New Relic, and Prometheus.
- Actuator web endpoints can be secured using Spring Security, much like any other endpoint in a Spring application.

Previous chapter
< Part 4. Deployed Spring

Next chapter
16 Administering Spring >